UNIVERSIDAD DON BOSCO

DIRECCIÓN DE EDUCACIÓN A DISTANCIA

Desarrollo de Aplicaciones Web con Software Interpretados en el Cliente DAW901 G01T (Virtual)

DOCENTE:

Alexander Alberto Siguenza Campos

ETAPA 3

Avance 2 Foro

SaborExpress

Equipo NovaTeam

Integrantes:

Nombre	Carné	Participación
Abner Isaí Díaz Rivera	DR261575	100%
Marlene Yamileth Pérez Alvarado	PA261400	100%
Elías Grande Beltrán	GB261371	100%
Jason Steven Durán López	DL261125	100%
Blanca Margarita Rivas Sandoval	RS260234	100%

FECHA DE ENTREGA:

08/11/2026

Integración de React y de la API Nominatim en SaborExpress

SaborExpress es una aplicación web de pedidos y entregas a domicilio para una pupusería. En las etapas anteriores se construyó con HTML, CSS y JavaScript sin framework y, en el Foro 1, se le agregaron librerías de interactividad. En el Foro 2 se reconstruyó con React (Meta Platforms, Inc., s.f.), la biblioteca de JavaScript para construir interfaces con componentes que el equipo eligió como framework del proyecto, y se integró una API externa real, Nominatim (Nominatim, s.f.), que convierte direcciones en ubicaciones y ubicaciones en direcciones. Este documento explica qué es cada tecnología, por qué se eligió, en qué parte del proyecto se aplica y cómo se verificó.

La aplicación quedó formada por doce pantallas, cuatro contextos de estado global, más de veinte componentes reutilizables y servicios que no dependen de React, y se verificó con 163 pruebas automáticas.

Contexto Técnico del Proyecto

La versión anterior cargaba cada página HTML con módulos de JavaScript y guardaba los datos en `localStorage` (Mozilla Developer Network [MDN], s.f.-b). En esta entrega el código nuevo vive en la carpeta `frontend` del mismo repositorio; el sitio anterior se conserva sin cambios para no romper los enlaces de las entregas pasadas. La aplicación se desarrolla con Vite (Vite, s.f.), que ofrece un servidor de desarrollo rápido y genera la versión de producción en la carpeta `dist`.

Se mantuvieron las reglas de negocio de las etapas anteriores: tarifa de envío fija de $1.00, zona de entrega de 5 km alrededor del local, tope de 20 unidades por producto, estados finales de pedido (Entregado y Cancelado) y dos sesiones independientes, una de cliente y otra

de administrador. Los datos siguen guardándose en el navegador; la base de datos en la nube queda para la entrega final.

Tecnologías Utilizadas

React

Qué es

React es una biblioteca de JavaScript de código abierto para construir interfaces de usuario a partir de componentes: funciones que reciben datos y devuelven la parte de la pantalla que les corresponde (Meta Platforms, Inc., s.f.). Cuando los datos cambian, React vuelve a dibujar solo lo necesario. Los hooks (`useState`, `useEffect`, `useMemo`, `useCallback` y `useContext`) permiten usar estado y efectos dentro de esas funciones. La versión utilizada es la 19.

Por Qué se Eligió

En la Etapa 1 el equipo decidió usar React y esa decisión se mantuvo. Frente a Angular, React tiene una curva de aprendizaje menor para un equipo que ya domina JavaScript, no obliga a adoptar TypeScript ni una estructura rígida, y se integra con las librerías que el proyecto ya usaba, como Chart.js, Leaflet y SweetAlert2. Además permite migrar el sitio por partes: cada integrante reconstruyó sus pantallas como componentes independientes.

Dónde se Aplica en SaborExpress

Toda la interfaz es React: doce pantallas en la carpeta `pages` y más de veinte componentes reutilizables, entre ellos las barras de navegación, los formularios, la ventana modal, la paginación, las gráficas y el mapa. Un ejemplo es `FormField`, que dibuja una etiqueta, un campo y su mensaje de error, y se reutiliza en el registro, el inicio de sesión, el checkout, el contacto y el panel del menú.

Vite

Vite es la herramienta que sirve el proyecto durante el desarrollo y lo compila para producción (Vite, s.f.). Divide el código en archivos que se descargan solo cuando se necesitan: la biblioteca de PDF únicamente al pedir un comprobante y las gráficas únicamente al abrir el panel de reportes. Los comandos principales son `npm run dev` para trabajar y `npm run build` para generar la versión publicada.

React Router

React Router gestiona las rutas de una aplicación de una sola página: cambia la pantalla sin recargar el navegador (React Router, s.f.). SaborExpress define las rutas de la tienda, de la cuenta y del panel de administración con tres diseños (`StoreLayout`, `AuthLayout` y `AdminLayout`), y carga cada página con `lazy`. El Código 1 muestra la ruta protegida del cliente: si no hay sesión, envía al inicio de sesión y recuerda a dónde volver.

Código 1

Ruta protegida del cliente

```
export function RutaCliente() {
  const { sesion } = useAuth();
  const ubicacion = useLocation();
  if (!sesion) {
    return <Navigate
to={`/login?redirect=${encodeURIComponent(ubicacion.pathname)}`} replace />;
  }
  return <Outlet />;
}
```

Nota. Fragmento de `components/layout/RutasProtegidas.jsx`.

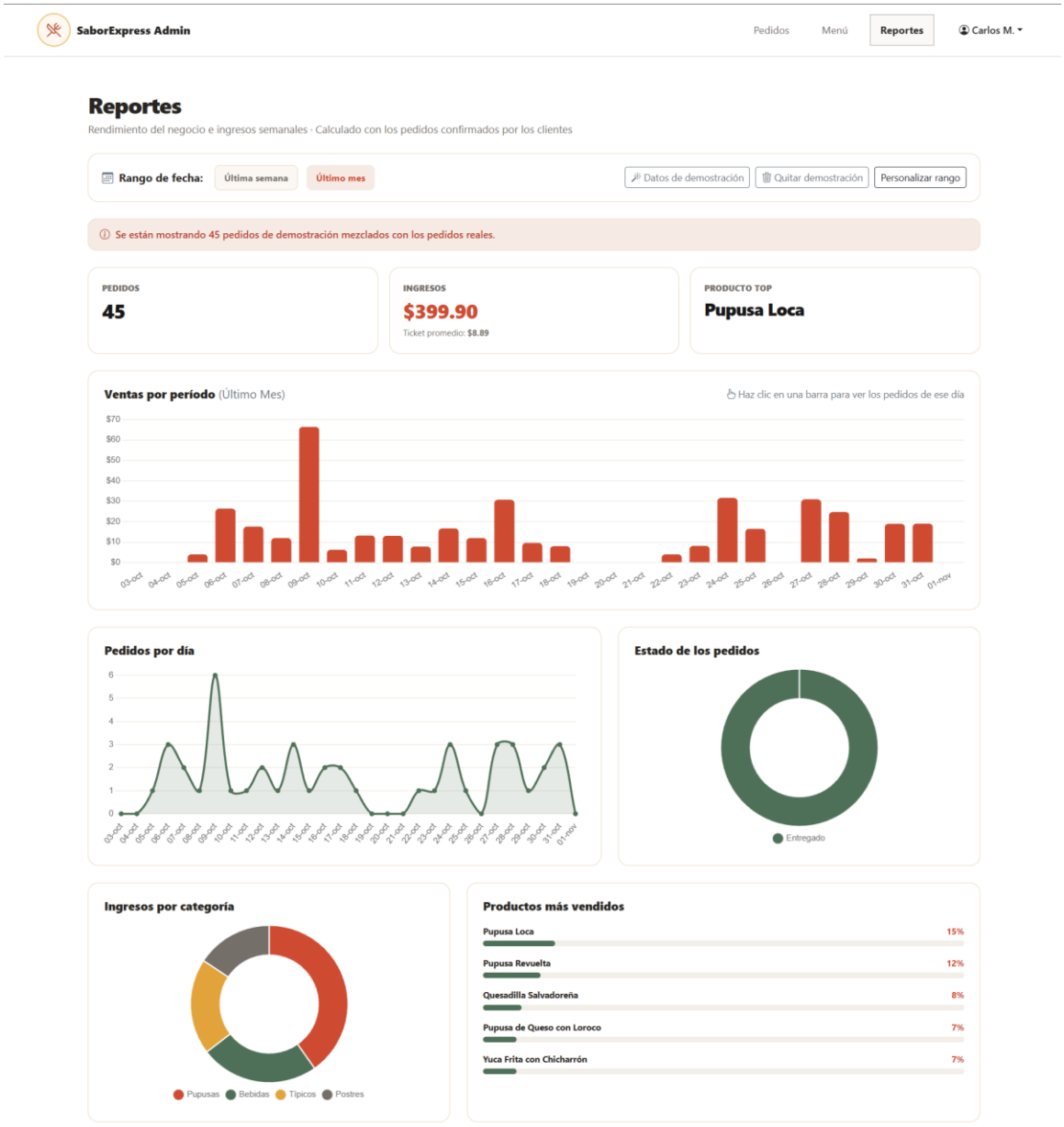
react-chartjs-2

react-chartjs-2 permite usar Chart.js como componentes de React (react-chartjs-2, s.f.), mientras que Chart.js dibuja las gráficas (Chart.js, s.f.). El panel de reportes usa cuatro: barras de ventas por día, línea de pedidos por día y dos donas, una de ingresos por categoría y otra del estado de los pedidos. Los datos se calculan con `useMemo` a partir de funciones puras, de modo

que las gráficas solo se actualizan cuando cambian los pedidos o el rango de fechas. Un clic en una barra abre el detalle de los pedidos de ese día. La Figura 1 muestra el panel.

Figura 1

Panel de reportes con las cuatro gráficas de react-chartjs-2



react-leaflet

React Leaflet ofrece los componentes de Leaflet (Agafonkin, s.f.) para usarlos en React (React Leaflet, s.f.). En el checkout, el componente `MapPicker` dibuja el mapa de OpenStreetMap, el círculo de 5 km de la zona de entrega, el local y un pin que se arrastra o se mueve tocando el mapa. La distancia entre el pin y el local se calcula con la fórmula del haversine (Sinnott, 1984) en `cobertura.js`, y el pedido solo se acepta si queda dentro de la zona.

SweetAlert2 y jsPDF como Paquetes

En el Foro 1 estas librerías se cargaban desde un CDN; ahora son dependencias del proyecto. SweetAlert2 (SweetAlert2, s.f.) se usa para las confirmaciones, los avisos y los formularios emergentes, y jsPDF (Parallax, s.f.) genera el comprobante del pedido, que se importa de forma dinámica solo cuando alguien lo pide. Así la aplicación no depende de servidores externos para funcionar, salvo el mapa y la API.

Arquitectura de Componentes

La Figura 2 muestra cómo se organiza la aplicación. `main.jsx` monta el enrutador y los proveedores de estado; `App.jsx` define las rutas; cada ruta usa un diseño y una página, y las páginas se construyen con componentes, hooks y servicios reutilizables. La Tabla 1 resume la responsabilidad de cada carpeta.

Tabla 1

Carpetas de la aplicación y su responsabilidad

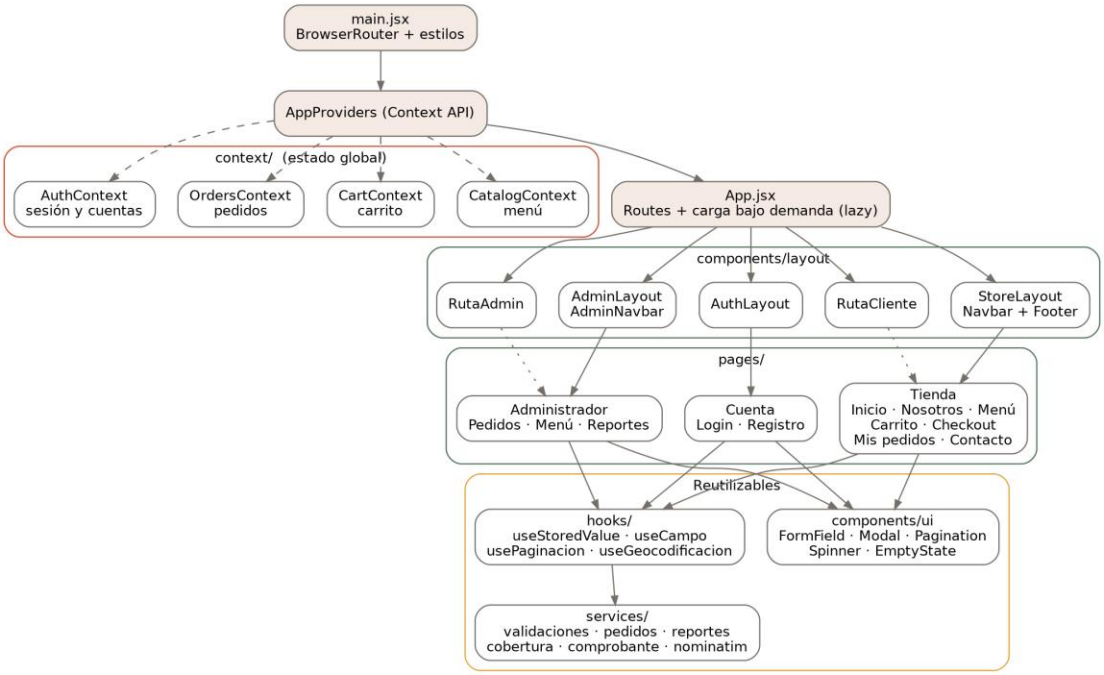
Carpeta	Responsabilidad	Ejemplos
`pages/`	Una pantalla por ruta	`MenuPage`, `CheckoutPage`, `ReportesPage`
`components/`	Piezas reutilizables, agrupadas por módulo	`FormField`, `Modal`, `MapPicker`

Carpeta	Responsabilidad	Ejemplos
`context/`	Estado global compartido	`AuthContext`, `CartContext`
`hooks/`	Lógica de React reutilizable	`useStoredValue`, `useCampo`
`services/`	Reglas de negocio sin React, que se prueban solas	`validaciones`, `reportes`, `nominatim`
`styles/`	Estilos del sitio	`sitio.css`, `app.css`
`test/`	Pruebas automáticas	`cuenta.test.jsx`, `nominatim.test.js`

Nota. Elaboración propia a partir del código del proyecto.

Figura 2

Arquitectura de componentes de la aplicación React



Nota. Elaboración propia.

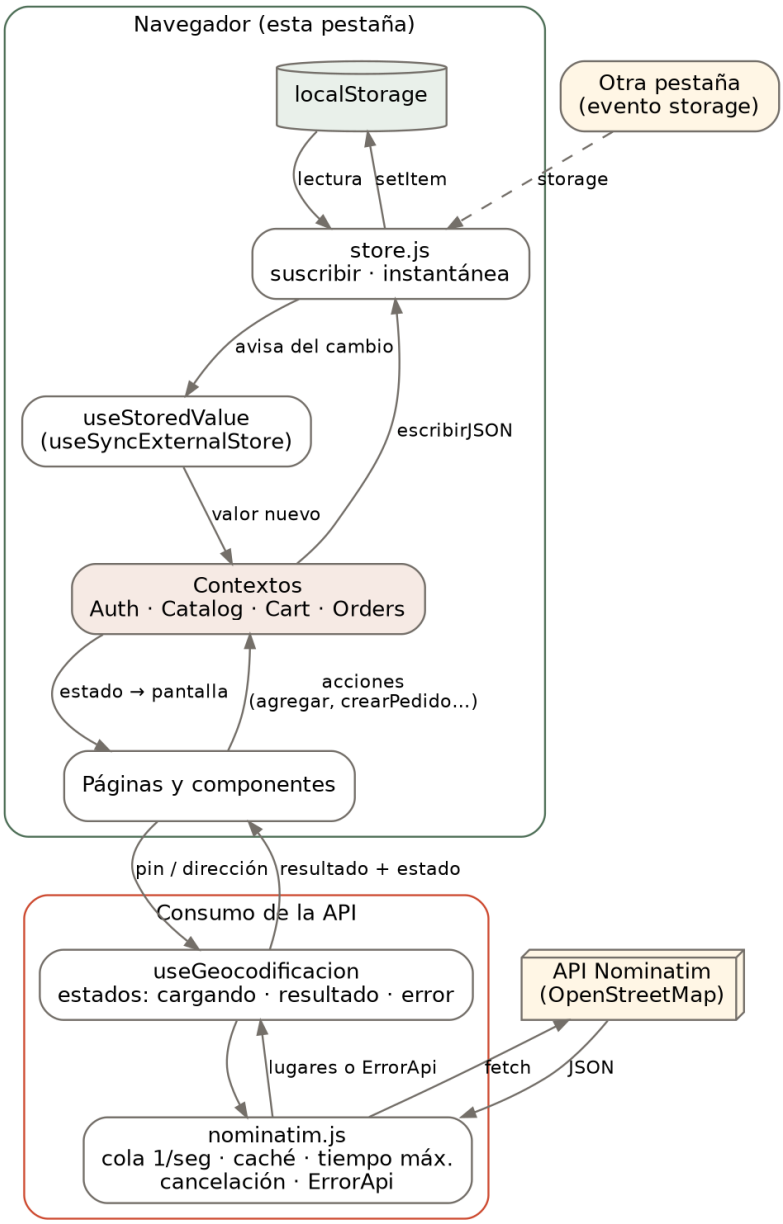
Manejo de Estado y Flujo de Datos

El estado global se maneja con la Context API de React: cuatro contextos (sesión, catálogo, carrito y pedidos) que cualquier componente lee con un hook (`useAuth`, `useCatalog`, `useCart` y `useOrders`). Los datos viven en `localStorage`, que no avisa a React cuando cambia. Para resolverlo, `store.js` mantiene una lista de suscriptores por clave y

el hook `useStoredValue` los conecta con `useSyncExternalStore`, la función de React pensada para leer datos externos. La Figura 3 resume el recorrido.

Figura 3

Flujo de datos entre el almacenamiento, el estado global y la interfaz



Nota. Elaboración propia.

Hay dos casos de actualización. En el primero, una escritura de esta pestaña: el contexto escribe el dato y avisa a quienes lo leen. En el segundo, una escritura de otra pestaña, que el navegador anuncia con el evento `storage` (Mozilla Developer Network [MDN], s.f.-c). Gracias a esto, cuando el administrador cambia el estado de un pedido, la pantalla del cliente se actualiza sin recargar, siempre que ambos estén en el mismo navegador.

Dos detalles evitan errores. Primero, la lectura devuelve exactamente el mismo objeto mientras el texto guardado no cambie (el Código 2); de lo contrario React volvería a dibujar sin parar. Segundo, las acciones del carrito leen el dato más reciente del almacenamiento y no el de la última pantalla dibujada, para que dos clics seguidos no se pisen.

Código 2

Lectura estable del almacenamiento

```
export function instantanea(clave, respaldo) {
  const crudo = leerCrudo(clave);
  const previo = memoria.get(clave);
  if (previo && previo.crudo === crudo) return previo.valor;
  let valor = respaldo;
  if (crudo !== null) {
    try { valor = JSON.parse(crudo); } catch { valor = respaldo; }
  }
  memoria.set(clave, { crudo, valor });
  return valor;
}
```

Nota. Fragmento de `services/store.js`.

Consumo de la API Nominatim

Qué es y para Qué se Usa

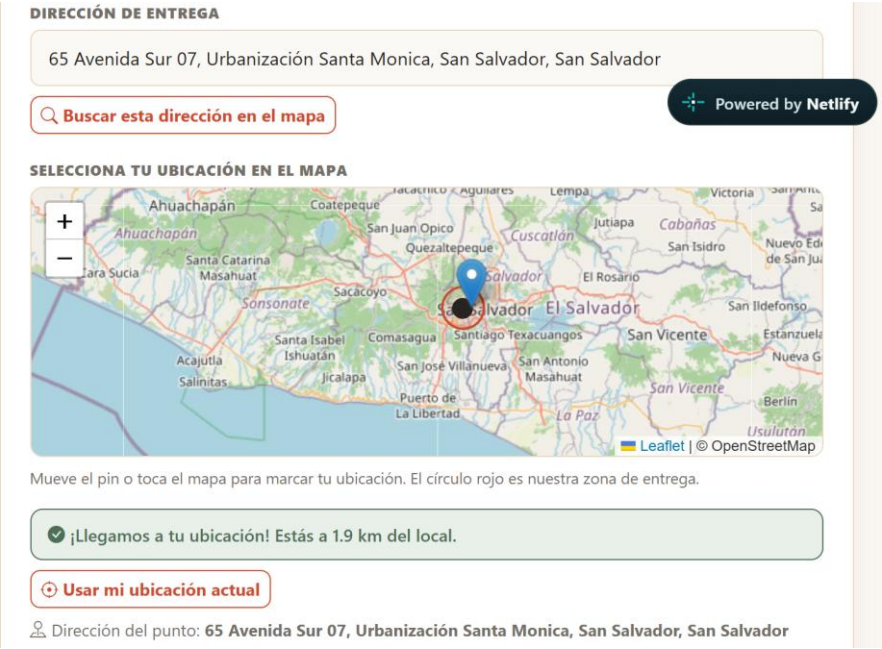
Nominatim es el servicio de geocodificación de OpenStreetMap: convierte una dirección escrita en coordenadas (búsqueda) y unas coordenadas en una dirección (geocodificación inversa) (Nominatim, s.f.). Devuelve datos reales en formato JSON y no necesita clave de API, por lo que no hay credenciales que proteger en el repositorio.

Cómo se Integró

En el checkout intervienen dos consultas. Si la persona escribe una dirección y pulsa el botón de buscar, la aplicación llama al punto de búsqueda (`/search`), limitado a El Salvador, y muestra hasta cinco sugerencias; al elegir una, el pin se mueve a esas coordenadas. Si la persona mueve el pin o usa su ubicación, la aplicación llama a `/reverse` y muestra la dirección encontrada, que rellena el campo si estaba vacío o se ofrece con el botón «Usar esta dirección». La Figura 4 muestra la pantalla.

Figura 4

Checkout con la dirección detectada por la API Nominatim



El código está en tres capas: `nominatim.js` se encarga de la comunicación con la API, `useGeocodificacion.js` entrega a la pantalla los estados de cargando, resultado y error, y `CheckoutPage.jsx` dibuja la interfaz.

Protecciones y Manejo de Errores

La política de uso de la instancia pública (OpenStreetMap Foundation, s.f.) pide, entre otras cosas, no superar una consulta por segundo y no usar la búsqueda como autocompletado. Por eso `nominatim.js` coloca las consultas en una cola con al menos 1.1 segundos entre ellas (el Código 3), guarda en memoria las respuestas repetidas, corta las consultas que tardan más de ocho segundos y permite cancelar la anterior con `AbortController` (Mozilla Developer Network [MDN], s.f.-a). La búsqueda de direcciones solo se lanza con el botón o con la tecla Enter, nunca mientras se escribe.

Código 3

Cola de peticiones con espera máxima

```
function esperarTurno(signal) {
  const turno = cola.then(async () => {
    const espera = Math.min(
      ESPERA_ENTRE_PETICIONES_MS,
      Math.max(0, ultimaPeticion + ESPERA_ENTRE_PETICIONES_MS - Date.now())
    );
    if (espera) await dormir(espera, signal);
    ultimaPeticion = Date.now();
  });
  cola = turno.catch(() => {});
  return turno;
}
```

Nota. Fragmento de `services/nominatim.js`.

Los fallos se convierten en un error propio con un tipo, y la pantalla muestra un mensaje claro para cada uno (la Tabla 2). En todos los casos la persona puede escribir la dirección a mano y seguir comprando.

Tabla 2

Errores de la API y mensaje que ve la persona

Tipo	Cuándo ocurre	Mensaje en pantalla
`red`	No hay internet o el servicio no responde	No hay conexión con el servicio de mapas. Revisa tu internet o escribe la dirección manualmente.

Tipo	Cuándo ocurre	Mensaje en pantalla
`tiempo`	Pasan más de ocho segundos	El servicio de mapas tardó demasiado en responder. Inténtalo de nuevo.
`limite`	El servicio responde con el código 429	Hay muchas consultas en este momento. Espera unos segundos e inténtalo de nuevo.
`servidor`	Cualquier otro error HTTP	El servicio de mapas no está disponible ahora. Puedes escribir la dirección manualmente.
`cancelado`	Se pidió otra consulta antes de que terminara la anterior	No se muestra ningún mensaje

Nota. Elaboración propia a partir del código del proyecto.

Limitaciones

La instancia pública no ofrece garantías de disponibilidad. Las pruebas automáticas simulan sus respuestas (éxito, sin resultados, códigos 429 y 503, falta de conexión y tiempo agotado), de modo que la verificación contra el servicio real depende de la prueba manual con conexión a internet.

Componentes Reutilizables

La reutilización se logró con componentes y con hooks propios. `useCampo` guarda el valor de un campo, lo valida al salir de él y mientras se corrige, y puede dar formato a lo que se escribe, por ejemplo el teléfono 7777-7777. `FormField` usa ese objeto para dibujar el campo con los estilos de validación de Bootstrap. La Tabla 3 resume las piezas más reutilizadas. La Figura 5 muestra el registro, donde se ven los mensajes de error y el medidor de contraseña; la Figura 6, el menú público, y la Figura 7, el panel de pedidos del administrador, que comparten la paginación.

Tabla 3

Componentes y hooks más reutilizados

Pieza	Qué hace	Dónde se reutiliza
`FormField` y `useCampo`	Campo con validación y mensaje de error	Registro, inicio de sesión, checkout, contacto y panel del menú

Pieza	Qué hace	Dónde se reutiliza
`Modal`	Ventana modal controlada por React	Rango personalizado de reportes y alta de productos
`Pagination` y `usePaginacion`	Paginación de listas	Menú público y panel de pedidos
`EmptyState` y `Spinner`	Mensajes de "sin datos" y de carga	Menú, Mis pedidos, página 404 y carga de páginas
`useStoredValue`	Lectura reactiva del almacenamiento	Los cuatro contextos de estado global

Nota. Elaboración propia a partir del código del proyecto.

Figura 5

Registro con errores de validación y medidor de contraseña



Figura 6

Menú público con búsqueda, filtros y paginación

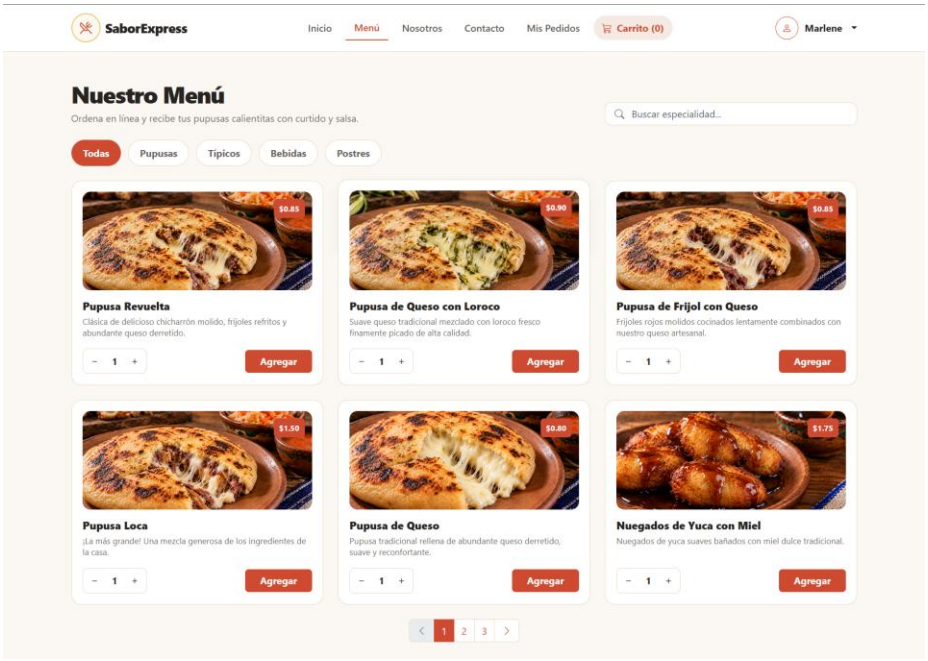
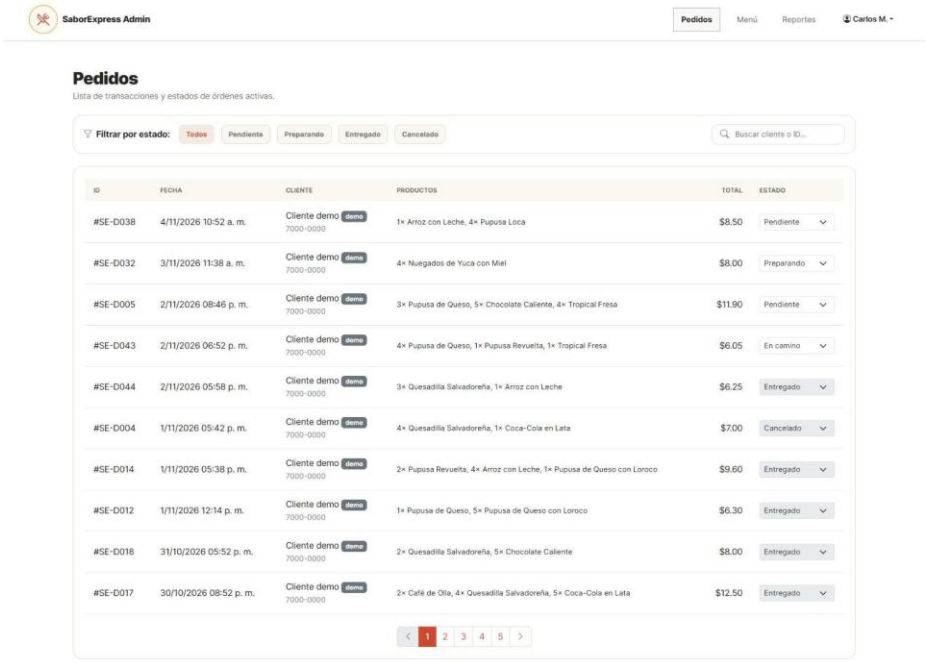


Figura 7

Panel de pedidos del administrador con filtros y estados



Pruebas Automáticas

Las pruebas usan Vitest (Vitest, s.f.) como ejecutor y React Testing Library (Testing Library, s.f.) para dibujar componentes y simular lo que hace una persona: escribir, pulsar y navegar. Los servicios sin React se prueban directamente; las pantallas se prueban dibujando la aplicación completa en una ruta. Las librerías que necesitan un navegador real (las gráficas y el mapa) y SweetAlert2 se reemplazan por dobles que exponen sus datos, y la API se simula con respuestas de éxito y de error. En total son 163 pruebas y todas pasan (la Tabla 4).

Tabla 4

Pruebas automáticas por archivo

Archivo	Qué verifica	Pruebas
`servicios.test.js`	Almacenamiento reactivo, validaciones, compra, catálogo, pedidos y autenticación	14
`base.test.jsx`	Contextos, navegación, sesión y sincronización entre pestañas	14
`rutas.test.jsx`	Rutas protegidas por rol	5
`menu-reportes.test.jsx`	Menú público y panel de reportes	23
`reportes.test.js`	Cálculos de reportes y datos de demostración	10
`cuenta.test.jsx`	Registro, inicio de sesión y mis pedidos	27
`contacto-admin.test.jsx`	Contacto y paneles de pedidos y de menú	25
`compra.test.jsx`	Carrito y checkout, incluida la API	31
`nominatim.test.js`	Cliente de la API: caché, cola, errores y cancelación	10
`comprobante.test.js`	PDF de una y de dos páginas	3
`cobertura.test.js`	Zona de cobertura	1
Total		163

Nota. Elaboración propia a partir del código del proyecto.

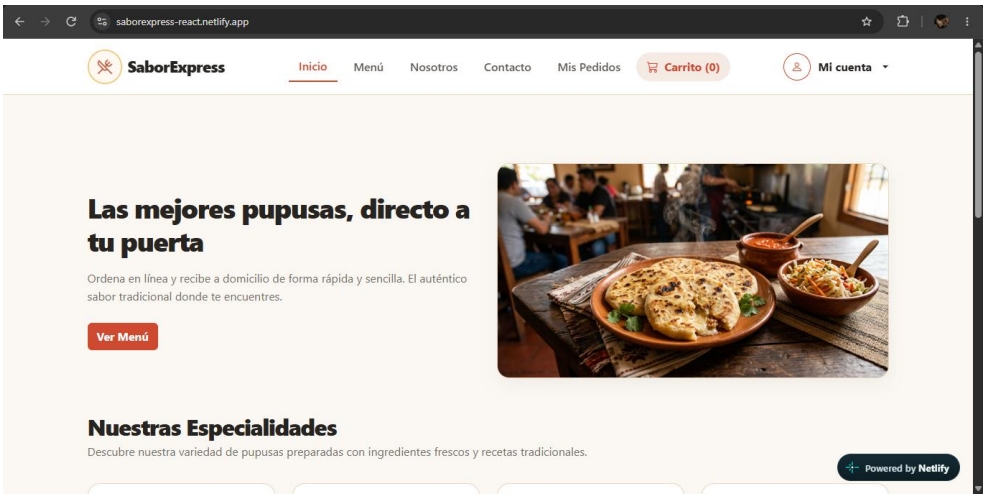
Durante el desarrollo, las pruebas encontraron dos fallas reales que se corrigieron: las acciones del carrito podían usar datos desactualizados y la espera de la API podía alargarse si el reloj del dispositivo retrocedía.

Despliegue

La aplicación se publica en Netlify (Netlify, s.f.), que descarga el repositorio, ejecuta `npm run build` en la carpeta `frontend` y publica la carpeta `dist`. Esa configuración está en el archivo `netlify.toml`. Como la aplicación es de una sola página, el archivo `_redirects` envía cualquier ruta a `index.html`; sin él, recargar una dirección como `/menu` produciría un error 404. Cada vez que se mezcla un cambio a la rama principal, Netlify publica de nuevo. La Figura 8 muestra el sitio publicado.

Figura 8

Aplicación publicada en Netlify



Resultados y Limitaciones

Se cumplieron los requisitos de la actividad: un framework integrado en todos los módulos, una API externa con datos reales, módulos secundarios funcionales, estado global y componentes reutilizables. Las limitaciones principales son cuatro. Los datos viven en el navegador, por lo que no se comparten entre dispositivos y el cliente y el administrador solo se ven entre sí en el mismo navegador. La autenticación es simulada: las contraseñas se guardan sin cifrar, algo aceptable para una demostración pero no para producción. La función de direcciones

depende de la instancia pública de Nominatim. Y las pruebas automáticas no comparan la apariencia, que se revisa manualmente.

Conclusiones y Recomendaciones

Reconstruir SaborExpress con React permitió separar la interfaz en piezas pequeñas y reutilizables, concentrar el estado en contextos y probar la lógica de forma automática, algo que en la versión anterior era difícil de lograr. El consumo de la API mostró que integrar un servicio externo exige más que una llamada: hay que cuidar los límites de uso, los tiempos de espera y los errores para que la aplicación siga siendo útil cuando el servicio falla.

Para la entrega final se recomienda reemplazar el almacenamiento del navegador por una base de datos en la nube (Firebase), que resolvería la sincronización entre dispositivos y la autenticación real, y agregar pruebas de extremo a extremo en un navegador real.

Enlaces del Proyecto

El código fuente de la aplicación y su historial de trabajo están en el repositorio de GitHub: https://github.com/abnerdiazz/SaborExpress-DAW

La aplicación publicada en Netlify puede visitarse en: https://saborexpress-react.netlify.app/

El video de demostración, en el que cada integrante muestra y explica su parte, puede verse en: https://drive.google.com/file/d/1_i5_3z2qiql3eq87jH1KtFvsDlK6kHYL/view?usp=drive_link

Referencias

Agafonkin, V. (s.f.). *Leaflet: An open-source JavaScript library for mobile-friendly interactive maps*. https://leafletjs.com/

Chart.js. (s.f.). *Chart.js documentation*. https://www.chartjs.org/docs/latest/

Meta Platforms, Inc. (s.f.). *React*. https://react.dev/

Mozilla Developer Network. (s.f.-a). *AbortController*. https://developer.mozilla.org/es/docs/Web/API/AbortController

Mozilla Developer Network. (s.f.-b). *Web Storage API*. https://developer.mozilla.org/es/docs/Web/API/Web_Storage_API

Mozilla Developer Network. (s.f.-c). *Window: storage event*. https://developer.mozilla.org/es/docs/Web/API/Window/storage_event

Netlify. (s.f.). *Netlify docs*. https://docs.netlify.com/

Nominatim. (s.f.). *Nominatim documentation*. https://nominatim.org/release-docs/latest/

OpenStreetMap Foundation. (s.f.). *Nominatim usage policy*. https://operations.osmfoundation.org/policies/nominatim/

Parallax. (s.f.). *jsPDF* [Software]. GitHub. https://github.com/parallax/jsPDF

react-chartjs-2. (s.f.). *react-chartjs-2*. https://react-chartjs-2.js.org/

React Leaflet. (s.f.). *React Leaflet*. https://react-leaflet.js.org/

React Router. (s.f.). *React Router documentation*. https://reactrouter.com/

Sinnott, R. W. (1984). Virtues of the haversine. *Sky and Telescope, 68*(2), 159.

SweetAlert2. (s.f.). *SweetAlert2: A beautiful, responsive, customizable, accessible replacement for JavaScript's popup boxes*. https://sweetalert2.github.io/

Testing Library. (s.f.). *React Testing Library*. https://testing-library.com/docs/react-testing-library/intro/

Vite. (s.f.). *Vite: Next generation frontend tooling*. https://vite.dev/

Vitest. (s.f.). *Vitest: Next generation testing framework*. https://vitest.dev/